

Project on NLP

UMSL CMP SCI 4300/5300

Requirements

1. Please read everything (including Additional Requirements) before starting to work on this project.
2. You are allowed to use AI tools such as ChatGPT to research and seek help for completing this assignment, as long as:
 - a. you write the report on your own, and
 - b. you do not rely on AI to complete the entire assignment at once.
3. All conversations with AI models (e.g., ChatGPT, Copilot) must be included in the report by sharing links to the conversations.
4. You are welcome to work in a group of up to two students. If you worked in a group, describe each person's contributions, how often you met, and how the work was divided.
5. You are expected to form a new group. In other words, if you have previously worked with certain classmates on earlier assignments, you cannot form a group with the same students again.
6. For any online work, such as tasks in Google Sheets or Google Colab, an accompanying PDF or file version must be submitted. Submitting a URL to Colab notebook is not enough.
7. Your project report *must be* prepared in Overleaf.
8. The first paragraph of your report should describe the problem you are solving.
9. Your report should also include a discussion of the challenges you faced while working on the project.
10. Include all evidence of your work, such as code and screenshots of outputs.
11. Your group may be asked to attend an in-person meeting to explain the project, with the grade being put on hold until then.
12. You must submit your work (code + report + others files) individually even if you worked in a group.

Project objective

The goal of this project is for you to learn about the “finding text similarity” problem in NLP, develop effective solutions, and analyze the limitations of the methods you create. You will be coding methods to find similarities between pairs of passages.

Getting ready

1. Learn an interesting application of finding text similarity by installing and trying the [EditGPT Google Chrome extension](#) for ChatGPT.

To determine **a**the clinical manifestations, treatments, and outcomes of SLE patients, the authors retrospectively studied 19 patients (aged **less than** <15 years) **who were** diagnosed with SLE at Thammasat University **H**Hospital; from January **0**1, 2002, **through** March 31, 2010. The mean age **i**was 12.9 +/- 1.6 year **old**s, **with a** mean follow-up of 3.3 +/- 2.6 years. Seventeen (89.5%) **of the** patients were female. **H**The clinical manifestations **included** hematological **involvement** (89.5%), dermatologic **involvement** (73.7%), and renal involvement (68.4%) **were the Clinical manifestations**.

2. Download the [Automated Essay Scoring 2.0](#) essay dataset by Learning Agency Lab, available for free in Kaggle. Note that the dataset may not have exactly 24,000 essays.

Method 1

1. Calculate similarities between all the 24,000 essays by comparing their word frequencies. You are welcome to use any existing algorithms based on word frequencies or code your own method. Your method has to be primarily based on word frequencies, not any advanced methods.
2. In your report, describe the steps involved in the algorithm, including any mathematical expressions.
3. Find the top five most similar pairs of essays, list their essays IDs, and one line preview of each essay. Any two essays that are exact duplicates should be excluded from your calculation. For example, essays that contain the word "PROPER_NAME" can be removed from the similarity calculation process. Many essays that have the word "PROPER_NAME" have another essay with the exact same text but this placeholder is replaced with someone's name.
4. What are some major practical limitations of this approach? Discuss a few pairs of essays where the similarity is low but the algorithm marks high similarity and two pairs of essays that are highly similar but the algorithm assigns a low score. You are welcome to use tools such as copyleaks.com to demonstrate your findings (see Figure 1). **This discussion is a key part of this project and a critical grading component.**
5. Discuss why the algorithm results in such limitations.
6. Discuss your code execution and show the execution time scales as you increase the number of essays.
7. Include all results in the report under the section "Method 1: Word frequency-based Approach"

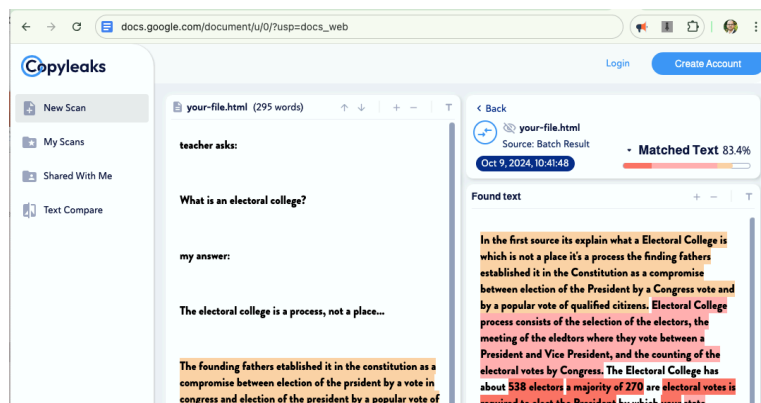


Figure 1. Copyleaks can help you check if a similarity score makes sense or not.

Method 2

Repeat all the steps in Method 1, using a different algorithm that is more accurate. By accurate, we mean a method that produces numbers that make more sense. For example, if the match percentage is 90%, the essays should indeed be similar. Include all results in the report under the section "Method 2: [Name of your method]."

Method 3

Repeat all the steps in Method 1, using a method that is faster than Method 1. The new method/algorithm has to be significantly different. Include all results in the report under the section "Method 3: [Name of your method]."

Additional requirements

At least of the two methods (Method 2 or 3) should be based on text vectorization. For example:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
vectorizer = TfidfVectorizer() # Create a TF-IDF vectorizer
```